

Serial communication of data via UART (Universal Asynchronous Receiver/Transmitter)

1 Paper's purpose

By the end of this first laboratory every student should have basic knowledge regarding serial communication of data via UART (Universal Asynchronous Receiver/Transmitter) and make practical use of (in asm programs) the special registers of this system.

2 Introduction

In this chapter will be presented:

- 2.1 Serial communications
- 2.2 UART1
- 2.3 Timer1
- 2.4 PuTTY

2.1 Serial communications

2.1.1 Parameters

Serial communication involves transforming the information stored as bytes into a series of consecutive bits consisting of data bits and control bits, transmitting this string of bits through a physical environment, and reconstituting, at reception, of the initial information.

Generally, symbols are transmitted, so the transmission frequency is called the baud rate, but in the particular case where the symbols are even bits, the baud rate is actually the bit rate.

Serial transmission also involves the introduction of some control bits, so that the original information can be reconstituted at the reception. Thus, the bits transmitted in serial communication can be:

- start bits
- data bits
- stop bits
- parity bits

2.1.2 UART

UART is a device that translates data between serial and parallel interfaces. Used in serial communication, UART converts data stored as bytes into strings of bits (data bits delimited by start and stop bits).

The UART standard defines the bits string structure as follows:

- a start bit (logical level 0)
- five to eight data bits
- an optional parity bit (sum of data bits)
- one, one and a half or two stop bits (logical level 1)

2.1.3 RS-232

Generally, the UART does not realize the effective data transmission to the physical environment. Usually, a serial interface is used to transform the voltage logic levels with which the UART works in the levels of the external signal (the signal that actually flows through the physical environment).

Such an interface is defined by the EIA RS-232 standard, this type of interface being also found on the development boards in the laboratory.

2.2 UART1

The C8051F040 microcontroller has two asynchronous transmission-reception systems, UART0 and UART1. In this paper, we will detail how UART1 works.

UART1 is a full-duplex, asynchronous serial port that can work in 8-bit UART and 9-bit UART modes defined in the 8051 UART standard.

UART1 has associated 2 special registers: Serial Control Register (SCON1) and Serial Data Buffer (SBUF1). The SBUF1 register is actually a buffer that interfaces the user with UART's physical transmission and reception registers. A write operation of this special register determines the start of a transmission, as the data is sent to the transmission shift register, while a reading operation of this register accesses the UART reception latch.

If the interruptions associated with UART1 are enabled, an interruption will be generated whenever a transmission is complete (TI1 bit of SCON1 register is set), or a data byte has been received (RI1 bit of SCON1 register is set). The interrupt flags (TI1 and RI1) are not reset by hardware, but must be reset of software after the cause of the interruption (full transmission / reception) has been determined. For more information about the SCON1 definition and the SBUF1 definition, refer to the microcontroller documentation (file C8051F04xRev1_4.pdf, pages 284 - 285).

2.2.1 UART1 operating modes

UART1 can work in one of the 8-bit UART modes, respectively 9-bit UART mode, depending on the logical value of the S1MODE bit of SCON1 special register.

2.2.1.1 8-bit mode

In this operating mode, an information byte is transformed into a 10-bit string as follows:

- the first bit is a start bit
- the next 8 bits are information bits, sorted starting with the least significant
- the last bit is a stop bit

The data is transmitted on the TX1 pin and is received on the RX1 pin. At the reception, the 8 data bits are stored in the SBUF1 register, and the ninth bit (the stop for this operating mode) is stored in the RB81 bit of SCON1 register.

The transmission starts when a byte is written in the SBUF1 register. The TI1 flag is automatically set at the end of each transmission (at the time of transmission of the stop bit). Data reception can begin at any time after setting the Receive Enable bit (bit 4 of SCON1 register). Once the stop bit has been received, the received byte is loaded from the physical receiving register in the SBUF1 register only if RI1 bit has a logical value of 0. All data received while this bit is 1 are ignored because the microcontroller considers the byte received beforehand has not yet been processed.

2.2.1.2 9-bit mode

In this operating mode, an information byte is transformed into a 11-bit string as follows:

- the first bit is a start bit
- the next 8 bits are information bits, sorted starting with the least significant
- the next bit is a programmable control bit
- the last bit is a stop bit

The type of the ninth bit is determined by the value of TB81 bit (bit 3 of SCON1 register). This bit can be set as parity bit or as signaling bit in multiprocessor communication. At the reception, this ninth bit is stored in RB81 bit of SCON1 register. Transmission and reception is done in the same way as in the 8-bit mode.

2.2.1.3 Baud-rate generation

UART1 baud-rate is generated by Timer 1 configured in Mode 2 (8-bit auto-reload). More information about Timer 1 can be found in chapter 2.3.1.

Transmission baud-rate generation:

- Timer 1 overflows (TL1) are used.
- Symbols transmission frequency is half the frequency of Timer 1 overflows (TL1).
- Reloading TL1 (with TH1 value) is determined by an overflow.

Reception baud-rate generation:

- The overflows of a Timer 1 copy (RX Timer) are used.
- Symbols reception frequency is half the frequency of overflows of this copy.
- RX Timer reloading (with TH1 value) is determined by an overflow or by the detection of a start bit – this allows the receiver to start at any time regardless of the Timer 1 status.

Baud rate generation is presented in Fig. 2.2.1.3.

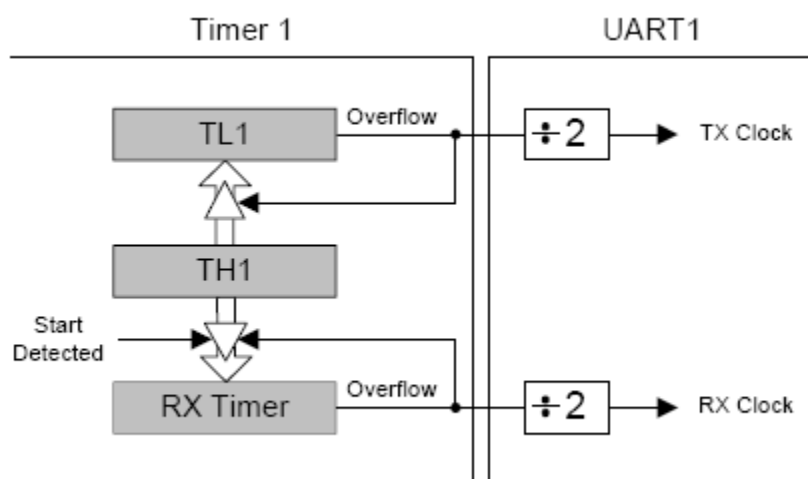


Fig. 2.2.1.3. baud-rate generation for UART1

The reload value of Timer 1 (TH1) must be set so that the overflows occur at twice the desired baud rate. Timer 1 can be synchronized with one of the following sources: SYSCLK, SYSCLK / 4, SYSCLK / 12, SYSCLK / 48, or with an external oscillator frequency divided by 8. The frequency of the symbols can be determined from the equation below, where T1CLK is the clock frequency with which Timer 1 is synchronized, and TH1 is the reload value of Timer 1.

$$\text{UartBaudRate} = 0.5 * \text{T1CLK} / (256 - \text{TH1})$$

Note: SYSCLK is the system clock frequency.

Tables Table 22.1 to Table 22.6 from the microcontroller documentation (file C8051F04xRev1_4.pdf, pages 286-288) set the T1CLK and TH1 values for different baud rates.

2.3 Timers

The C8051F040 microcontroller has five 16-bit timers: Timer 0, Timer 1 (specific to 8051 standard) and Timer 2, Timer 3, Timer 4 (specific to this microcontroller and commonly used in analogue-to-digital converters).

2.3.1 Timer 1

- is a timer / counter implemented with a 16-bit register accessible by bytes such as: a lower byte (TL1) and a higher byte (TH1).
- can be synchronized with one of the following clock sources:
 - system clock
 - system clock with frequency divided by 4, 12 or 48
 - an external oscillator with frequency divided by 8depending on the T1M, SCA1 and SCA0 bits configuration in CKCON register. For more information about the CKCON definition, refer to the microcontroller documentation (C8051F04xRev1_4.pdf, pg. 295).
- can work as a counter by counting the falling fronts of an external signal applied to a pin of a general purpose port. It is not mandatory for the signal to be periodic, but its frequency should not exceed one quarter of the internal clock frequency, and the signal levels should last for at least two periods of the internal clock.
- TCON register is used to activate the timer, as well as to check the status of the timer. For more information on the TCON definition, refer to the microcontroller documentation (C8051F04xRev1_4.pdf, pg. 293).
- T1M1 - T1M0 bits in TMOD register set the timer operating mode in one of three modes:
 - mode 0 – 13-bit counter/timer
 - mode 1 – 16-bit counter/timer
 - mode 2 – 8-bit auto-reload counter/timer

For more information about the TMOD definition, refer to the microcontroller documentation (file C8051F04xRev1_4.pdf, p. 294).

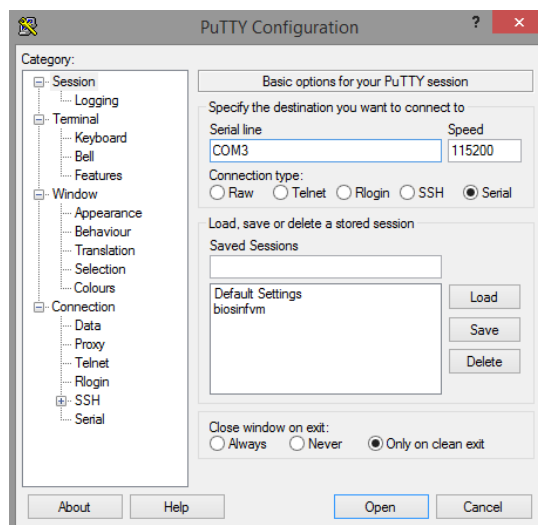
2.3.1.1 Timer 1 working in Mode 2 (8-bit auto-reload)

In this mode, the value stored in TL1 is incremented at each clock of the clock source with which the Timer is synchronized, and TH1 retains the reload value. At the transition time from 0xFF to 0x00 (overflow), TL1 register is reloaded with the value stored in TH1, and bit TF1 (bit 7 of TCON register) is set. If Timer 1 interrupts are enabled, an interruption is generated when TF1 bit is set. The value stored in TH1 is not changed during operation.

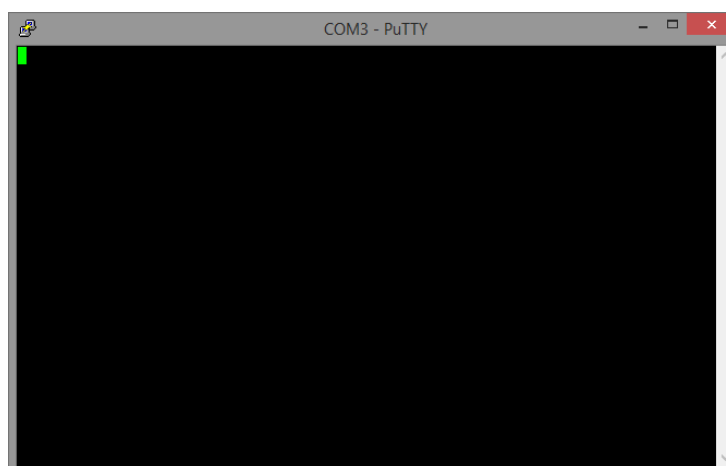
2.4 PuTTY

PuTTY a terminal emulator that helps communicate with other devices, using various protocols (SSH, Serial, etc.). Establish a serial connection between the microcontroller and the computer, by executing the following steps:

- Start PuTTY, using the Desktop shortcut: 
- Configure the connection such as: connection type (Serial), baud rate value (Speed =115200), port name used for communication (Serial line = COM1 for computers or Serial line = COM3 for laptops):



- Press Open and the terminal used for writing (in the case of transmission) or used to see (in the case of reception) will be prompted :



Application example

2.5 Requirements

Implement a serial communication system between the computer and the microcontroller C8051F040. Use PuTTY and a serial cable to transmit an ASCII character between A and Z to the microcontroller, and it will respond by sending back ASCII characters starting with the received character and ending with the Z character.

Note: American Standard Code for Information Interchange (short ASCII) is one of the oldest encoding standards. The ASCII table represents a 0-255 number (8-bit encoded) and some graphic symbols (including characters 0-9, a-z, A-Z). Certain ASCII codes are control characters rather than graphic symbols (for example: end of text, backspace, line feed, form feed, etc.). The ASCII table is presented in Appendix 4.2.

2.6 Proposed solution

2.6.1 Hardware module description

We will briefly present the implementation of a system that uses the C8051F040 development board. The development board (serial port labeled J5) is connected to the computer (COM1 port) via a serial cable.

2.6.2 Algorithm description

Note that the algorithm and the source code follow the general structure presented in Paper 1, chapter 3.2.2.

The solution presented will be composed of several procedures: `main`, `init`, `sendByte` and `receiveByte`.

The algorithm consists of a loop that calls itself infinitely, labeled `start`. There are several operations inside this loop:

- `receiveByte` subroutine is called, thus receiving a byte and storing it in the accumulator,
 - the value of the accumulator is compared (using subtraction) with the ASCII codes of characters "A" and "Z", and if the received character is outside of this range, a jump to `error` label is performed.
- Notes:
- the received value is previously saved in register B, since the value in register A is changed after these operations,
 - the jump to `error` label is done if CY flag (Carry Flag) in PSW register is set. This flag is automatically set if at subtraction a "loan" is needed, meaning if the second operand is higher.
- next, the ASCII code of the received character is stored in R0 register, and R1 register is initialized with the number of characters to be transmitted by UART1. R0 is used to store the ASCII characters of the characters to be displayed in PuTTY, and R1 plays the counter in the display loop;
 - enter the transmission loop labeled `loop`, whose purpose is to transmit the appropriate characters (starting with the character received and ending with the Z character) as follows:
 - `sendByte` subroutine is called, thus transmitting the code stored in R0
 - R0 value is incrementing (thus passing to the next character),
 - display loop counter is decremented, and if it's not yet zero, it jumps to a new display iteration.
 - if the `error` label is reached, the same registers as above (R0 and R1) are used to send three "?" characters.

The main loop (`main`) calls the initialization subroutine (`init`), then enters the infinite loop labeled `start`. This area follows exactly the steps of the algorithm described above, and finally calls itself, creating an infinite cycle.

The transmission subroutine - has the role of transmitting through UART1 the code stored in register R0

- writes the value from R0 to SBUF1 register, thus commanding the transmission of that byte,
- waits for TI1 flag to be set, signaling the end of the transmission,
- resets TI1 flag to prepare it for the next transmission.

The reception subroutine - has the role of receiving a byte through UART1 and storing it in the accumulator

- waits for RI1 flag to be set, indicating that a byte has been received,
- stores the received byte (in SBUF1) in the accumulator,
- resets RI1 flag to release the receive buffer (from now on it can receive a new data byte).

The initialization subroutine

- disables the interrupt system
 - please see Paper 1 for details.
- disables the watchdog timer mechanism
 - please see Paper 1 for details.
- enables the crossbar
 - in this application UART1 activation is needed, therefore both the bit that enables the crossbar and the bit that enables UART1 will be set.
 - for details on the crossbar configuration and the definition of XBR2 register, refer to the microcontroller documentation (C8051F04xRev1_4.pdf, p. 206, p. 216).
- selects the system clock and sets the frequency division factor
 - in this application, the SoC's internal oscillator with a base frequency of 24.5MHz is used and the dividing factor is set to one. The reasons for this frequency selection will be presented at the next point.

- for details on the OSCICN definition and the CLKSEL definition refer to the microcontroller documentation (file C8051F04xRev1_4.pdf, p. 176, p. 177).
- configures Timer 1
 - as described in chapter 2.2.1.3. Timer 1 is used to generate baud rate for UART 1. Considering this, it must be configured in 8-bit auto-reload mode.
 - in this application, the baud rate must be set to 115200 bits/s. Taking into account that the internal clock frequency has already been chosen (24.5MHz), the values T1CLK and TH1, from the equation presented in chapter 2.2.1.3. must be defined, according to Table 22.1 of the microcontroller documentation (file C8051F04xRev1_4.pdf, pages 286-288). It follows that T1CLK must be the system clock, and the reload value of Timer 1 must be 0x96.
 - the code labeled initTimer1 selects Timer 1 operating mode (8-bit auto-reload), sets the reload value of TH1 (0x96), selects Timer 1 sync source (system clock) and finally activates Timer 1.
- configures UART 1
 - UART 1 is set to work in 8-bit mode, as we do not need the transmission to be extremely safe (so we will not use the checksum mechanism presented in chapter 2.2.1.2.). Reception is also enabled by setting the REN1 bit.

3 Exercises

1. Create a new project using the file Z:\Lab8051\Laborator3\serialCommunication.asm following the steps presented in Paper 1, Section 2.2.
2. Execute the application.
3. Modify the program so that after the transmission of the appropriate characters, the application will pass to a new line, at the beginning of the row.
4. Modify the program so that after the transmission of the appropriate characters, the application will pass to a new page.
5. Modify the program so that the characters are transmitted at a minimum interval of one second.
6. Modify the program to receive a digit and transmit back the digits i , $i + 1$, ..., 9 (where i is the received digit).
7. Modify the program to display your first and last name on each row of PuTTY.
8. Modify the initializations for UART 1 and Timer 1 to make the transmission in 9-bit mode (with the ninth bit as parity bit) at a baud rate of 9600 bits/s.
9. Modify the program from point 6 so that it does not retransmit to PuTTY, the received data will be displayed on the LED bar as follows:
 - if digit i is received, $i = 1..9$, turn on the first i LEDs from left to right.
 - if something else is received, turn on the 10th LED (first on the right).Indications:
 - the LED bar is connected to the microcontroller according to APPENDIX 5.2 of Paper 2.
10. Modify the previous program so that the received data will be displayed on the 7-segment display as follows:
 - if display i is received, $i=0..9$, turn on digit i .
 - if something else is received, display an error symbol (E).Indications:
 - the 7-segment display is connected to the microcontroller through port 4 (P4) according to APPENDIX 5.2 of Paper 2.
 - each of the subroutines presented in APPENDIX 5.3 of Paper 2 turn on one digit, respectively the error symbol (E) on the 7-segment display.

4 Appendixes

4.1 SerialCommunication source code

```

;-----
;
;
; FILE NAME   : serialCommunication.asm
; TARGET MCU  : C8051F040
; DESCRIPTION : UART1 serial communication program.
;
; NOTES:
;-----

#include (c8051f040.inc)                ; Include register definition file.

;-----
; RESET and INTERRUPT VECTORS
;-----

                cseg      AT 0x0000        ; Reset Vector
                ljmp      main            ; Locate a jump to the start of code
                                           ;at the reset vector.
;-----
; MAIN PROGRAM CODE SEGMENT
;-----

mainCodeSeg     segment      CODE

                rseg      mainCodeSeg     ; Switch to this code segment.
                using     0               ; Specify register bank for the following
                                           ; program code.

main:           acall      init           ; Initialization of all used SFR's
                mov       SFRPAGE, #UART1_PAGE

start:          call       receiveByte    ; Start the algorithm
                mov       B, A           ; Receive a character through UART1
                clr       C              ; Compare the value in A with #0x41
                subb      A, #0x41        ; and jump to error if lower
                jc         error          ;
                mov       A, #0x5A        ; Compare the value 0x5A with the value
                subb      A, B           ; stored in B and jump to error if lower
                jc         error          ;

                mov       R0, B           ; Store in R0 the previously received ASCII code
                mov       R1, A           ; Init the loop counter
                inc        R1

                loop:      call       sendByte    ; Transmit the character with
                inc        R0            ; the ASCII code stored in R0
                djnz      R1, loop        ; Increment the ASCII code stored in R0
                sjmp       start          ; Decrement R1 and jump to loop if not zero
                                           ; Start over again

                error:      mov       R0, #0x3F    ; Store in R0 the ASCII code of "?"
                mov       R1, #0x03        ; Init the loop counter
                errorLoop: call       sendByte
                djnz      R1, errorLoop    ; Decrement R1 and jump if not zero
                sjmp       start          ; Start over again

;-----
; FUNCTION CODE
;-----

init:           clr        EA            ; Disable global interrupts
                mov       WDTCN, #0xDE     ; Disable Watch Dog Timer
                mov       WDTCN, #0xAD
                mov       SFRPAGE, #CONFIG_PAGE ; Use SFRs in the configuration Page
                mov       XBR2, #0x44      ; Enable the crossbar and UART 1

initOscillator: mov       CLKSEL, #0x00
                mov       OSCICN, #0xC3

```

```
initTimer1:      mov      SFRPAGE, #TIMER01_PAGE      ; Use SFRs in Timer 1 Page
                 mov      TMOD, #0x20                ; Use Timer 1 in 8-bit auto-reload mode
                 mov      TH1, #0x96                 ; Set the reload value
                 mov      CKCON, #0x10               ; Timer 1 will use the system clock
                 mov      TCON, #0x40                ; Enable Timer 1

initUART1:       mov      SFRPAGE, #UART1_PAGE ; Use SFRs in UART 1 Page
                 mov      SCON1, #0x10             ; Use UART 1 in 8-bit mode

                 ret

sendByte:        mov      SBUF1, R0                  ; Transmit the byte stored in R0
                 jnb      TI1, $                     ; Wait for the end of transmission
                 clr      TI1                        ; Clear the end of transmission flag
                 ret

receiveByte:     jnb      RI1, $                     ; Wait for the end of reception
                 mov      A, SBUF1                   ; Copy the received data in the accumulator
                 clr      RI1                        ; Clear the end of reception flag
                 ret

;-----
; End of file.

END
```

4.2 ASCII Table

Dec	Hx	Oct	Char	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr
0	0	000	NUL (null)	32	20	040	 Space		64	40	100	@ @		96	60	140	` <b>`	
1	1	001	SOH (start of heading)	33	21	041	! !		65	41	101	A A		97	61	141	a a	
2	2	002	STX (start of text)	34	22	042	" "		66	42	102	B B		98	62	142	b b	
3	3	003	ETX (end of text)	35	23	043	# #		67	43	103	C C		99	63	143	c c	
4	4	004	EOT (end of transmission)	36	24	044	$ \$		68	44	104	D D		100	64	144	d d	
5	5	005	ENQ (enquiry)	37	25	045	% %		69	45	105	E E		101	65	145	e e	
6	6	006	ACK (acknowledge)	38	26	046	& &		70	46	106	F F		102	66	146	f f	
7	7	007	BEL (bell)	39	27	047	' '		71	47	107	G G		103	67	147	g g	
8	8	010	BS (backspace)	40	28	050	((		72	48	110	H H		104	68	150	h h	
9	9	011	TAB (horizontal tab)	41	29	051	))		73	49	111	I I		105	69	151	i i	
10	A	012	LF (NL line feed, new line)	42	2A	052	* *		74	4A	112	J J		106	6A	152	j j	
11	B	013	VT (vertical tab)	43	2B	053	+ +		75	4B	113	K K		107	6B	153	k k	
12	C	014	FF (NP form feed, new page)	44	2C	054	, ,		76	4C	114	L L		108	6C	154	l l	
13	D	015	CR (carriage return)	45	2D	055	- -		77	4D	115	M M		109	6D	155	m m	
14	E	016	SO (shift out)	46	2E	056	. .		78	4E	116	N N		110	6E	156	n n	
15	F	017	SI (shift in)	47	2F	057	/ /		79	4F	117	O O		111	6F	157	o o	
16	10	020	DLE (data link escape)	48	30	060	0 0		80	50	120	P P		112	70	160	p p	
17	11	021	DC1 (device control 1)	49	31	061	1 1		81	51	121	Q Q		113	71	161	q q	
18	12	022	DC2 (device control 2)	50	32	062	2 2		82	52	122	R R		114	72	162	r r	
19	13	023	DC3 (device control 3)	51	33	063	3 3		83	53	123	S S		115	73	163	s s	
20	14	024	DC4 (device control 4)	52	34	064	4 4		84	54	124	T T		116	74	164	t t	
21	15	025	NAK (negative acknowledge)	53	35	065	5 5		85	55	125	U U		117	75	165	u u	
22	16	026	SYN (synchronous idle)	54	36	066	6 6		86	56	126	V V		118	76	166	v v	
23	17	027	ETB (end of trans. block)	55	37	067	7 7		87	57	127	W W		119	77	167	w w	
24	18	030	CAN (cancel)	56	38	070	8 8		88	58	130	X X		120	78	170	x x	
25	19	031	EM (end of medium)	57	39	071	9 9		89	59	131	Y Y		121	79	171	y y	
26	1A	032	SUB (substitute)	58	3A	072	: :		90	5A	132	Z Z		122	7A	172	z z	
27	1B	033	ESC (escape)	59	3B	073	; ;		91	5B	133	[[		123	7B	173	{ {	
28	1C	034	FS (file separator)	60	3C	074	< <		92	5C	134	\ \		124	7C	174	| 	
29	1D	035	GS (group separator)	61	3D	075	= =		93	5D	135	]]		125	7D	175	} }	
30	1E	036	RS (record separator)	62	3E	076	> >		94	5E	136	^ ^		126	7E	176	~ ~	
31	1F	037	US (unit separator)	63	3F	077	? ?		95	5F	137	_ _		127	7F	177	 DEL	

128	�	144	�	161	�	177	�	193	�	209	�	225	�	241	�
129	�	145	�	162	�	178	�	194	�	210	�	226	�	242	�
130	�	146	�	163	�	179	�	195	�	211	�	227	�	243	�
131	�	147	�	164	�	180	�	196	�	212	�	228	�	244	�
132	�	148	�	165	�	181	�	197	�	213	�	229	�	245	�
133	�	149	�	166	�	182	�	198	�	214	�	230	�	246	�
134	�	150	�	167	�	183	�	199	�	215	�	231	�	247	�
135	�	151	�	168	�	184	�	200	�	216	�	232	�	248	�
136	�	152	�	169	�	185	�	201	�	217	�	233	�	249	�
137	�	153	�	170	�	186	�	202	�	218	�	234	�	250	�
138	�	154	�	171	�	187	�	203	�	219	�	235	�	251	�
139	�	156	�	172	�	188	�	204	�	220	�	236	�	252	�
140	�	157	�	173	�	189	�	205	�	221	�	237	�	253	�
141	�	158	�	174	�	190	�	206	�	222	�	238	�	254	�
142	�	159	�	175	�	191	�	207	�	223	�	239	�	255	�
143	�	160	�	176	�	192	�	208	�	224	�	240	�		